Министерство образования Республики Беларусь

Учреждение образования

БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина Операционные системы и системное программирование

ОТЧЁТ
к лабораторной работе
на тему

ВЗАИМОДЕЙСТВИЕ ПРОЦЕССОВ: ОБМЕН ДАННЫМИ

Вариант 26

Выполнил Д.А. Скробот

Проверил Р.А. Шабанович

Минск 2026

СОДЕРЖАНИЕ

Введение 3
1 Архитектура программного комплекса 4
2 Выбранный механизм IPC 7
3 Формат передаваемых сообщений 9
4 Синхронизация и API 11
5 Реализация и обработка ошибок 13
6 Результаты испытаний 16
7 Результаты измерения производительности 18
8 Использование ИИ 19
Заключение 20

ВВЕДЕНИЕ

Вариант 26: *Логирование через Mailslot*.

Требуется реализовать программный комплекс, в котором три клиента отправляют сообщения одному серверу. Каждое сообщение содержит идентификатор процесса, порядковый номер, время отправки и текст. Сервер принимает сообщения и записывает их. В качестве механизма IPC должен использоваться Mailslot.

Цель работы – изучить механизмы межпроцессного взаимодействия в операционной системе и получить практические навыки организации обмена данными между независимыми процессами. В ходе работы необходимо освоить создание нескольких взаимодействующих процессов, разработку собственного протокола сообщений, использование механизма Mailslot и его аналога в Linux, обработку отключения и аварийного завершения участников, измерение характеристик IPC, а также корректное освобождение системных ресурсов.

Дополнительно требуется:

– обеспечить работу одного управляющего процесса и трёх одновременно работающих клиентов;

– передавать в каждом сообщении версию протокола, тип, PID отправителя, идентификатор запроса, размер и полезную нагрузку;

– вести журнал основных операций обмена с временем, PID, `RUN_ID` и номером варианта;

– предусмотреть штатное завершение по сообщению `STOP`;

– измерить время передачи не менее 1000 сообщений двух размеров: около 64 байт и не менее 4 КБ;

– сравнить Mailslot и Named Pipe и обосновать выбор механизма с учётом требований к надёжности.

1 АРХИТЕКТУРА ПРОГРАММНОГО КОМПЛЕКСА

Комплекс собран в один исполняемый файл `logger`, который запускается с указанием роли и обязательного идентификатора запуска `RUN_ID`:

```
./build/logger run <RUN_ID>
./build/logger bench <RUN_ID>
./build/logger server <RUN_ID>
./build/logger client <RUN_ID> <INDEX>
./build/logger stop <RUN_ID>
```

В режиме `run` управляющий процесс `MANAGER` создаёт четыре дочерних процесса: сервер и три клиента. Все процессы являются отдельными процессами Linux и не используют общие глобальные переменные для обмена данными. Единственный канал передачи данных – датаграммный сокет `AF_UNIX`, привязанный к файлу `/tmp/oeasp_lr3_<RUN_ID>.sock`.

Менеджер удаляет устаревший файл сокета, запускает сервер и ждёт появления этого файла. Только после готовности сервера запускаются клиенты. После завершения всех клиентов менеджер отправляет серверу сообщение `STOP`, дожидается его завершения и удаляет файл сокета.

Модульная структура программы приведена в таблице 1.1.

Таблица 1.1 – Модули программы

Модуль	Назначение
`protocol.h`	константы, типы сообщений и структуры протокола
`logging.c`	журнал, форматирование времени, вывод ошибок
`transport.c`	путь сокета, адрес `sockaddr_un`, отправка датаграмм
`server.c`	создание серверного сокета, приём и проверка сообщений
`server_handlers.c`	обработка сообщений `LOG`, `BENCH`, `BENCH_END`
`client.c`	обычный клиент, отправляющий 13 сообщений
`bench_client.c`	клиент замера производительности
`control.c`	отправка сообщения `STOP`
`manager.c`	запуск процессов, ожидание и очистка
`process.c`	`waitpid`, ожидание готовности сокета
`main.c`	разбор аргументов и выбор роли

Таблица 1.2 – Процессы комплекса

Роль	Количество	Назначение
`MANAGER`	1	разбирает аргументы, удаляет старый сокет, запускает сервер и клиентов, ждёт их, отправляет `STOP`, удаляет сокет

Продолжение таблицы 1.2

Роль	Количество	Назначение
SERVER	1	создаёт и привязывает сокет, принимает датаграммы, проверяет протокол, пишет журнал и файл received_<RUN_ID>.log, завершается по STOP
CLIENT#1–CLIENT#3	3	создают собственные сокеты и отправляют по 13 сообщений LOG с PID, порядковым номером, временем и текстом
BENCH	1 в режиме bench	отправляет по 1103 сообщения размером 64 и 4096 байт, измеряет время

Всего в режиме run работают пять процессов: менеджер, сервер и три клиента. Каждый процесс при старте выводит роль, PID, номер варианта и RUN_ID. Пример строки:

```
[23:21:17.564] PID=449787 VARIANT=26 RUN_ID=9101 ROLE=CLIENT#1 START client_index=1
```

Схема взаимодействия процессов приведена на рисунке 1.1. Сплошными серыми стрелками показано создание процессов через fork, синими – передача данных через sendto (для BENCH – штриховой), красной штриховой – управляющее сообщение STOP.

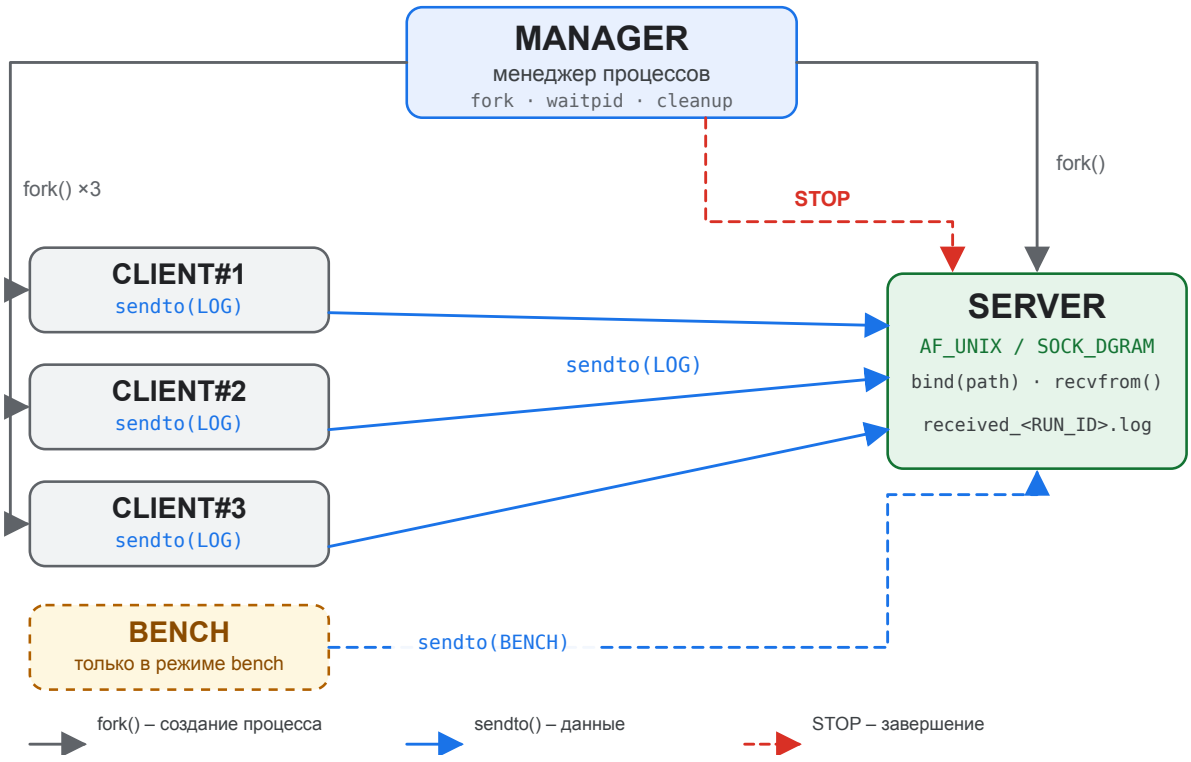


Рисунок 1.1 – Схема взаимодействия процессов

Процесс BENCH создаётся только в режиме bench; в обычном режиме run менеджер запускает три клиента. Последовательность работы:

1 менеджер удаляет устаревший файл сокета и создаёт процесс сервера;

2 сервер вызывает socket, setsockopt и bind, после чего файл сокета становится виден;

3 менеджер ждёт появления файла сокета и запускает три клиента;

4 клиенты отправляют сообщения через sendto, сервер принимает их через recvfrom;

5 менеджер ждёт завершения клиентов и отправляет STOP;

6 сервер завершает цикл приёма, закрывает сокет и файл журнала, удаляет файл сокета;

7 менеджер дожидается сервера, удаляет сокет и выводит итог.

2 ВЫБРАННЫЙ МЕХАНИЗМ IPC

В Windows Mailslot – это односторонний механизм передачи сообщений без установления соединения. Сервер создаёт mailslot с помощью `CreateMailslot`, клиент открывает его через `CreateFile` и пишет сообщения функцией `WriteFile`; сервер читает их через `ReadFile`. Mailslot сохраняет границы сообщений, может использоваться для широковещательной рассылки, но не гарантирует доставку.

В Linux Mailslot отсутствует. В качестве аналога выбран датаграммный сокет `socket(AF_UNIX, SOCK_DGRAM)`, привязанный к пути в файловой системе. Клиент вызывает `sendto`, сервер – `recvfrom`. Такой сокет, как и Mailslot, не требует соединения и сохраняет границы сообщений. В Linux датаграммы `AF_UNIX` в пределах одной машины доставляются надёжно, а порядок сообщений от одного отправителя сохраняется; однако в протоколе этой работы подтверждения доставки не используются, поэтому по модели обмена решение соответствует Mailslot: отправитель не ждёт ответа сервера.

Сравнение Mailslot и Named Pipe приведено в таблице 2.1.

Таблица 2.1 – Сравнение Mailslot и Named Pipe

Свойство	Mailslot	Named Pipe
Соединение	не устанавливается	устанавливается перед обменом
Направление	одностороннее	одно- или двустороннее
Границы сообщений	сохраняются	зависит от режима: byte/message
Надёжность	доставка не гарантируется	гарантированная доставка
Широковещание	поддерживается	не поддерживается
Разграничение доступа	ограниченное	через параметры безопасности
Типичное применение	журналы, уведомления	запрос-ответ, управление

Выбор механизма должен определяться требованиями к надёжности. Если потеря отдельного сообщения недопустима или нужен ответ сервера, следует использовать Named Pipe либо потоковый сокет `AF_UNIX` с подтверждениями. Если же сообщения представляют собой односторонний поток журнальных записей и потеря допустима, подходит Mailslot: он проще, не требует установления соединения и позволяет нескольким отправителям работать независимо. В данной работе выбран именно такой сценарий, но для повышения устойчивости добавлены тайм-аут сервера, проверка кодов завершения процессов и штатное сообщение `STOP`.

Таблица 2.2 – Используемые IPC-объекты

Объект	Где создаётся	Назначение	Освобождение
серверный сокет	SERVER	приём датаграмм по пути /tmp/oeasp_lr3_<RUN_ID>.sock	close, unlink
клиентские сокеты	CLIENT#1–CLIENT#3, BENCH	отправка сообщений через sendto	close
файл сокета	SERVER при bind	адрес для связи процессов	unlink сервером и менеджером
файл журнала	SERVER	запись принятых сообщений в received_<RUN_ID>.log	fclose
процессы	MANAGER через fork	параллельная работа клиентов и сервера	waitpid

Разделяемая память, мьютексы и семафоры не используются: сервер однопоточный, а данные передаются копированием через сокет. Синхронизация достигается порядком запуска процессов, очередью датаграмм в сокете и ожиданием завершения потомков. Тайм-аут SO_RCVTIMEO не позволяет серверу ждать STOP бесконечно, если участники завершились аварийно.

3 ФОРМАТ ПЕРЕДАВАЕМЫХ СООБЩЕНИЙ

Сообщение состоит из заголовка фиксированного размера и полезной нагрузки. Все поля имеют фиксированную ширину, указатели в структурах отсутствуют. Структуры приведены в листинге 3.1.

Листинг 3.1 – Структуры протокола

```
typedef struct {
    uint32_t version;
    uint32_t messageType;
    uint32_t sourcePid;
    uint32_t requestId;
    uint32_t payloadSize;
} MessageHeader;

typedef struct {
    int64_t sentSec;
    int64_t sentNsec;
    uint32_t seq;
    uint32_t textLen;
    char text[128];
} LogPayload;

typedef struct {
    uint32_t smallSent;
    uint32_t largeSent;
    uint64_t smallNs;
    uint64_t largeNs;
} BenchEndPayload;
```

Заголовок занимает 20 байт. Поле `payloadSize` содержит фактическую длину полезной нагрузки, а не размер массива `text[128]`. Для обычного сообщения `payloadSize = 24 + textLen`; при `textLen = 39` полезная нагрузка занимает 63 байта, а вся датаграмма – 83 байта. В сообщении `STOP` полезная нагрузка отсутствует, `payloadSize = 0`. В сообщении `BENCH_END` передаётся структура `BenchEndPayload` размером 24 байта.

Поля сообщения:

– `version` – версия протокола, равна 1;

– `messageType` – тип сообщения;

– `sourcePid` – PID отправителя;

– `requestId` – идентификатор запроса; для клиента вычисляется как `index * 1000 + seq`, для `STOP` используется значение 999999;

– `payloadSize` – размер полезной нагрузки в байтах;

– `sentSec`, `sentNsec` – время отправки в секундах и наносекундах;

– `seq` – порядковый номер сообщения клиента;

– `textLen`, `text` – длина и текст сообщения.

Текст дополняется точками до 39 байт, чтобы параметр `STUDENT_ID` влиял на размер сообщения. Порядок байтов – native, так как отправитель и получатель работают на одной машине.

Таблица 3.1 – Типы сообщений

Тип	Отправитель	Получатель	Payload	Ответ
LOG	CLIENT#1–CLIENT#3	SERVER	PID, seq, время отправки, текст	–
STOP	MANAGER	SERVER	пусто; payloadSize = 0	–
BENCH	BENCH	SERVER	датаграмма 64 или 4096 байт	–
BENCH_END	BENCH	SERVER	количество отправленных сообщений и время двух серий	–

Отдельного типа ERROR нет: ошибки обнаруживаются локально, выводятся в журнал и не передаются другому процессу. Ответы не используются, поскольку выбранный механизм односторонний.

4 СИНХРОНИЗАЦИЯ И API

Синхронизация построена на порядке создания процессов и на сообщениях протокола:

1 менеджер удаляет устаревший файл сокета и создаёт процесс сервера;

2 менеджер ждёт появления файла сокета: цикл из 250 попыток с интервалом 20 мс проверяет `stat` и `S_ISSOCK`, а также завершение сервера через `kill(pid, 0)`;

3 после появления сокета менеджер запускает три клиента;

4 каждый клиент отправляет сообщения и завершается; менеджер ждёт каждый процесс через `waitpid`;

5 только после завершения клиентов менеджер отправляет `STOP`, поэтому все вызовы `sendto` уже завершены и датаграммы находятся в очереди сервера;

6 сервер обрабатывает сообщения в одном потоке; очередь сокета сохраняет порядок поступления датаграмм;

7 при получении `STOP` сервер закрывает сокет, файл журнала и удаляет файл сокета.

Мьютексы и семафоры не требуются: отсутствует общая изменяемая память, а сервер обрабатывает запросы последовательно. Если клиент завершился аварийно, `waitpid` сообщает об этом менеджеру; менеджер всё равно отправляет `STOP`, чтобы сервер не остался в ожидании. Если аварийно завершился сам сервер, клиенты получают ошибку `sendto`, а менеджер фиксирует ненулевой код завершения потомка.

Для замера производительности клиент сначала отправляет все сообщения серии, затем `BENCH_END`. Сервер запоминает время первого и последнего сообщения каждого размера и выводит результаты после получения `BENCH_END`.

Так как работа выполнена на Linux/POSIX, в таблице 4.1 приведены использованные функции Linux/POSIX и их аналоги в Windows API.

Таблица 4.1 – Использованные функции API

Linux/POSIX	Назначение	Аналог Windows API
`socket(AF_UNIX, SOCK_DGRAM)`	создание датаграммного сокета	`CreateMailslot`, `CreateFile`
`bind`	привязка сокета к пути	`CreateMailslot`
`sendto`	отправка датаграммы	`WriteFile`
`recvfrom`	приём датаграммы	`ReadFile`
`setsockopt`	тайм-аут приёма (`SO_RCVTIMEO`)	`SetMailslotInfo`, `WaitForSingleObject`
`close`	закрытие дескриптора	`CloseHandle`
`unlink`	удаление файла сокета	`DeleteFile`
`fork`	создание процесса	`CreateProcess`
`waitpid`	ожидание завершения потомка	`WaitForSingleObject`

Продолжение таблицы 4.1

Linux/POSIX	Назначение	Аналог Windows API
`kill(pid, 0)`	проверка существования процесса	`OpenProcess`, `GetExitCodeProcess`
`stat`	проверка файла сокета	`GetFileAttributes`
`clock_gettime`	измерение времени	`QueryPerformanceCounter`
`localtime_r`, `strftime`	форматирование времени	`GetLocalTime`, `GetDateFormat`
`errno`, `strerror`	диагностика ошибок	`GetLastError`, `FormatMessage`

5 РЕАЛИЗАЦИЯ И ОБРАБОТКА ОШИБОК

Для системных ошибок выводится имя функции, код `errno` и текстовое описание `strerror(errno)`. Обрабатываются следующие ситуации:

– невозможность создать сокет, привязать его к пути или установить тайм-аут;

– невозможность отправить сообщение: при отсутствии сервера `sendto` возвращает `ENOENT`, при завершённом сервере с оставшимся файлом сокета – `ECONNREFUSED`;

– ошибка чтения: `recvfrom` повторяется при `EINTR`; при `EAGAIN`/`EWOULDBLOCK` сервер сообщает об истечении тайм-аута 10 секунд и завершается с ошибкой;

– короткая датаграмма, датаграмма больше буфера (обнаруживается через `MSG_TRUNC`), несовпадение фактического размера и `payloadSize`;

– неизвестная версия протокола, неизвестный тип сообщения, неверная длина текста;

– `STOP` с ненулевой полезной нагрузкой;

– аварийное завершение дочернего процесса: `waitpid` проверяет код выхода и сигнал;

– очистка ресурсов: закрытие сокетов и файла журнала, удаление файла сокета.

Проверка некорректных сообщений дала следующий результат:

```
ERROR short message size=3
ERROR version=99 requestId=1
ERROR unknown type=77 requestId=2 size=0
ERROR size mismatch requestId=3 declared=5 actual=0
```

После обработки ошибочных датаграмм сервер продолжил работу и корректно завершился по `STOP`. Штатное завершение выполняется сообщением `STOP`; принудительное завершение процессов (`TerminateProcess`, `SIGKILL`) в нормальном сценарии не используется. Вызов `SIGTERM` возможен только при аварийной очистке, если сервер не создал сокет за отведённое время.

Ниже приведены сокращённые фрагменты.

Листинг 5.1 – Создание серверного сокета

```
static int open_server_socket(const char *path)
{
    struct sockaddr_un address;
    struct timeval timeout;
    int fd;

    fill_address(&address, path);
    fd = socket(AF_UNIX, SOCK_DGRAM, 0);
    if (fd < 0) {
        fatal_errno("socket");
    }

    timeout.tv_sec = SERVER_TIMEOUT_SEC;
    timeout.tv_usec = 0;
    if (setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO,
                   &timeout, sizeof(timeout)) < 0) {
```

```
        close(fd);
        fatal_errno("setsockopt");
    }

    if (bind(fd, (struct sockaddr *)&address,
             address_length(&address)) < 0) {
        close(fd);
        fatal_errno("bind");
    }
    return fd;
}
```

Листинг 5.2 – Проверка принятого сообщения

```
received = recvfrom(fd, datagram, sizeof(datagram),
                    MSG_TRUNC, NULL, NULL);
if ((size_t)received < sizeof(header)) {
    journal_error("ERROR short message size=%zd", received);
    continue;
}

memcpy(&header, datagram, sizeof(header));
payload_size = (uint32_t)received - (uint32_t)sizeof(header);

if (header.version != MSG_VERSION) {
    journal_error("ERROR version=%u requestId=%u",
                  header.version, header.requestId);
    continue;
}
if (header.payloadSize != payload_size) {
    journal_error("ERROR size mismatch");
    continue;
}
if (message_type_name(header.messageType) == NULL) {
    journal_error("ERROR unknown type=%u", header.messageType);
    continue;
}
dispatch_message(datagram, &header, payload_size,
                 log_file, &stopping);
```

Листинг 5.3 – Отправка сообщений клиентом

```
for (uint32_t seq = 1; seq <= CLIENT_MESSAGES; ++seq) {
    MessageHeader header;
    LogPayload payload;
    struct timespec sent;
    uint32_t request_id = client_index * 1000U + seq;
    size_t payload_size =
        offsetof(LogPayload, text) + LOG_TEXT_LEN;

    clock_gettime(CLOCK_REALTIME, &sent);
    payload.sentSec = sent.tv_sec;
    payload.sentNsec = sent.tv_nsec;
    payload.seq = seq;
    payload.textLen = LOG_TEXT_LEN;

    header.version = MSG_VERSION;
    header.messageType = MSG_LOG;
    header.sourcePid = (uint32_t)getpid();
    header.requestId = request_id;
    header.payloadSize = (uint32_t)payload_size;

    if (send_datagram(fd, &server_addr, datagram,
```

```
                        sizeof(header) + payload_size) < 0) {
            journal_error("ERROR func=sendto requestId=%u",
                        request_id);
            return EXIT_FAILURE;
        }
        journal("SEND requestId=%u type=LOG size=%zu",
                request_id, sizeof(header) + payload_size);
}
```

Листинг 5.4 – Отправка сообщения STOP

```
int send_stop(void)
{
    MessageHeader header;
    int fd = socket(AF_UNIX, SOCK_DGRAM, 0);
    header.version = MSG_VERSION;
    header.messageType = MSG_STOP;
    header.sourcePid = (uint32_t)getpid();
    header.requestId = 999999;
    header.payloadSize = 0;
    if (send_datagram(fd, &address, &header, sizeof(header)) < 0) {
        close(fd);
        return -1;
    }
    close(fd);
    return 0;
}
```

6 РЕЗУЛЬТАТЫ ИСПЫТАНИЙ

Ниже приведена сокращённая запись журнала одного полного запуска RUN_ID=9101; показаны все роли, отправка и приём сообщений, завершение и итог.

Листинг 6.1 – Журнал одного полного запуска (сокращён)

```
[23:21:17.543] PID=449783 VARIANT=26 RUN_ID=9101 ROLE=MANAGER START mode=run
    clients=3 messages_per_client=13 student_id=103
[23:21:17.543] PID=449783 VARIANT=26 RUN_ID=9101 ROLE=MANAGER FORK role=SERVER pid=449784
[23:21:17.543] PID=449784 VARIANT=26 RUN_ID=9101 ROLE=SERVER START
    listening path=/tmp/oeasp_lr3_9101.sock timeout=10s log=received_9101.log
[23:21:17.563] PID=449783 VARIANT=26 RUN_ID=9101 ROLE=MANAGER SERVER READY pid=449784
    path=/tmp/oeasp_lr3_9101.sock
[23:21:17.564] PID=449783 VARIANT=26 RUN_ID=9101 ROLE=MANAGER FORK role=CLIENT#1 pid=449787
[23:21:17.564] PID=449787 VARIANT=26 RUN_ID=9101 ROLE=CLIENT#1 START client_index=1
    messages=13 text_len=39
[23:21:17.564] PID=449787 VARIANT=26 RUN_ID=9101 ROLE=CLIENT#1 SEND requestId=1001
    type=LOG size=83
[23:21:17.564] PID=449783 VARIANT=26 RUN_ID=9101 ROLE=MANAGER FORK role=CLIENT#2 pid=449788
[23:21:17.564] PID=449788 VARIANT=26 RUN_ID=9101 ROLE=CLIENT#2 START client_index=2
    messages=13 text_len=39
[23:21:17.564] PID=449788 VARIANT=26 RUN_ID=9101 ROLE=CLIENT#2 SEND requestId=2001
    type=LOG size=83
[23:21:17.564] PID=449783 VARIANT=26 RUN_ID=9101 ROLE=MANAGER FORK role=CLIENT#3 pid=449789
[23:21:17.564] PID=449789 VARIANT=26 RUN_ID=9101 ROLE=CLIENT#3 START client_index=3
    messages=13 text_len=39
[23:21:17.564] PID=449789 VARIANT=26 RUN_ID=9101 ROLE=CLIENT#3 SEND requestId=3001
    type=LOG size=83
[23:21:17.564] PID=449784 VARIANT=26 RUN_ID=9101 ROLE=SERVER RECEIVE requestId=1001
    type=LOG size=63 from_pid=449787 seq=1 sent=23:21:17.564
    text="client=1 seq=1 variant=26.............."
[23:21:17.564] PID=449784 VARIANT=26 RUN_ID=9101 ROLE=SERVER RECEIVE requestId=2001
    type=LOG size=63 from_pid=449788 seq=1 sent=23:21:17.564
    text="client=2 seq=1 variant=26.............."
[23:21:17.565] PID=449784 VARIANT=26 RUN_ID=9101 ROLE=SERVER RECEIVE requestId=3001
    type=LOG size=63 from_pid=449789 seq=1 sent=23:21:17.564
    text="client=3 seq=1 variant=26.............."
[23:21:17.564] PID=449787 VARIANT=26 RUN_ID=9101 ROLE=CLIENT#1 DONE sent=13
[23:21:17.564] PID=449788 VARIANT=26 RUN_ID=9101 ROLE=CLIENT#2 DONE sent=13
[23:21:17.564] PID=449789 VARIANT=26 RUN_ID=9101 ROLE=CLIENT#3 DONE sent=13
[23:21:17.565] PID=449783 VARIANT=26 RUN_ID=9101 ROLE=MANAGER SEND requestId=999999
    type=STOP size=20
[23:21:17.565] PID=449784 VARIANT=26 RUN_ID=9101 ROLE=SERVER RECEIVE requestId=999999
    type=STOP size=0 from_pid=449783
[23:21:17.565] PID=449784 VARIANT=26 RUN_ID=9101 ROLE=SERVER STOPPED result=OK
[23:21:17.565] PID=449783 VARIANT=26 RUN_ID=9101 ROLE=MANAGER CHILD DONE role=SERVER pid=449784
[23:21:17.565] PID=449783 VARIANT=26 RUN_ID=9101 ROLE=MANAGER SUMMARY failures=0 result=OK
```

Проверялись два случая: отсутствие сервера и аварийное завершение уже запущенного сервера. В первом случае клиент не может открыть путь сокета, во втором файл сокета остаётся, но получатель уже завершён. Оба случая приводят к ошибке sendto и ненулевому коду возврата клиента.

```
$ ./build/logger client 9201 1
[23:21:39.265] PID=450313 VARIANT=26 RUN_ID=9201 ROLE=CLIENT#1 START
client_index=1 messages=13 text_len=39
[23:21:39.265] PID=450313 VARIANT=26 RUN_ID=9201 ROLE=CLIENT#1 ERROR
func=sendto requestId=1001 errno=2 (No such file or directory)
exit=1
$ ./build/logger server 9202 &
$ kill -9 <SERVER_PID>
$ ./build/logger client 9202 1
[23:21:39.576] PID=450327 VARIANT=26 RUN_ID=9202 ROLE=CLIENT#1 START
client_index=1 messages=13 text_len=39
[23:21:39.576] PID=450327 VARIANT=26 RUN_ID=9202 ROLE=CLIENT#1 ERROR
```

```
func=sendto requestId=1001 errno=111 (Connection refused)
exit=1
```

Сервер также защищён от бесконечного ожидания: если клиенты или менеджер завершились аварийно и `STOP` не поступил, `recvfrom` завершается по `SO_RCVTIMEO` через 10 секунд, сервер выводит ошибку и освобождает ресурсы. Менеджер проверяет код завершения каждого потомка; при сигнале или ненулевом коде он продолжает очистку и пытается отправить `STOP`; ошибка отправки учитывается как сбой.

7 РЕЗУЛЬТАТЫ ИЗМЕРЕНИЯ ПРОИЗВОДИТЕЛЬНОСТИ

Замер выполнялся в режиме `./build/logger bench <RUN_ID>`. Клиент отправлял 1103 сообщения размером 64 байта и 1103 сообщения размером 4096 байт. Время измерялось монотонным таймером `clock_gettime(CLOCK_MONOTONIC)` на клиенте и на сервере. Для каждой серии выполнено три запуска; в таблице 7.1 приведены времена серверной стороны и медианные значения.

Таблица 7.1 – Результаты измерения производительности

Тест	Размер	Сообщений	Три замера, мс	Медиана, мс	Медиана, сообщений/с	Пропускная способность, МБ/с
1	64 байта	1103	4.862; 6.318; 7.136	6.318	174574	11.17
2	4096 байт	1103	4.229; 6.286; 6.748	6.286	175466	718.72

Клиентские измерения совпадают с серверными в пределах долей миллисекунды: медиана для 64 байт составила 6.426 мс, для 4096 байт – 6.315 мс. Потери сообщений отсутствовали: `client_sent = server_received = 1103` в каждом запуске.

Количество сообщений в секунду для обоих размеров примерно одинаково (около 175 тысяч), поскольку при отправке датаграмм основную роль играет число системных вызовов, а не объём данных. Пропускная способность в байтах различается более чем в 60 раз: для сообщений 64 байта она составляет около 11 МБ/с, для 4096 байт – около 719 МБ/с. Это объясняется тем, что накладные расходы на одну датаграмму амортизируются большим объёмом полезной нагрузки. Результаты зависят от загрузки машины и планировщика, поэтому приведены медианы трёх запусков.

8 ИСПОЛЬЗОВАНИЕ ИИ

При выполнении работы использовался DeepSeek как справочный инструмент. ИИ не определял архитектуру комплекса и не заменял проверку программы: выбранный механизм IPC, структура процессов, формат сообщений и сценарии завершения были сформулированы и проверены самостоятельно. С помощью ИИ уточнялись отдельные теоретические вопросы и шаблоны для рутинных участков кода.

ИИ применялся для следующих задач:

– сравнение Mailslot и Named Pipe и выбор аналога Mailslot в Linux;
– объяснение флага `MSG_TRUNC` при приёме датаграмм;
– шаблон форматирования времени для строки журнала;
– шаблон вывода результатов замера производительности;
– объяснение поведения `SO_RCVTIMEO` при отсутствии сообщений.

Использованные промпты:

– «Объясни, чем Mailslot отличается от Named Pipe в Windows и какой механизм POSIX/Linux можно использовать как аналог Mailslot для односторонней передачи сообщений»;

– «Как работает флаг MSG_TRUNC в recvfrom и как определить, что датаграмма не поместилась в буфер»;

– «Как на POSIX получить время с миллисекундами и отформатировать его для строки журнала»;

– «Как оформить результаты замера IPC: количество сообщений, общее время и сообщений в секунду»;

– «Почему серверу нужен SO_RCVTIMEO и как обработать аварийное завершение клиента».

К рутинным фрагментам, для которых использовались шаблоны, относятся форматирование времени в `logging.c` (функции `vjournal` и `format_timestamp`), заполнение адреса `sockaddr_un` в `transport.c` и формирование итоговой строки замера в `bench_client.c`. Эти фрагменты были перепроверены, адаптированы под текущий протокол и протестированы. Работоспособность подтверждена запусками `run` и `bench`, тестами с некорректными сообщениями и сценариями отсутствующего и аварийно завершённого сервера.

ЗАКЛЮЧЕНИЕ

В результате выполнения работы реализован комплекс из пяти процессов: менеджер, сервер и три клиента. В качестве аналога Windows Mailslot на Linux использован датаграммный сокет `AF_UNIX` с типом `SOCK_DGRAM`, привязанный к файлу `/tmp/oeasp_lr3_<RUN_ID>.sock`. Клиенты отправляют сообщения через `sendto`, сервер принимает их через `recvfrom` и записывает в журнал и файл `received_<RUN_ID>.log`.

Реализованы собственный протокол с версией, типом, PID, `requestId`, размером и полезной нагрузкой, проверка некорректных сообщений, тайм-аут приёма, обработка аварийного завершения процессов и штатное завершение по сообщению `STOP`. Все дескрипторы и файлы закрываются, файл сокета удаляется.